

Lab sheet 14**Submission Due: 24.07.2024****Title:** HR Management System - 1**Aims:**

- Create Laravel project
- Connect with database
- Implement APIs

Tasks:

- Create new project
- Start Laravel's local development server
- Create a database
- Add database to the project
- Run migrations
- Create a model
- Add variables to the model
- Update migrations
- Create a controller
- Implement CRUD operation
- Implement routes
- Test APIs

Activities :

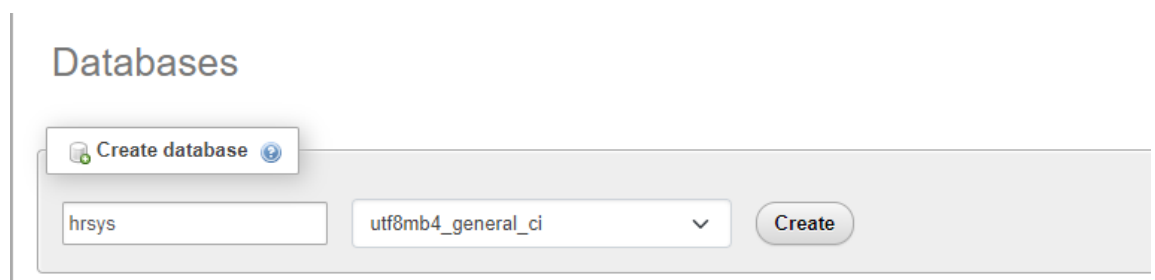
1. Create new project

composer create-project laravel/laravel hr-management-system

2. Start Laravel's local development server

cd hr-management-system**php artisan serve**

3. Create a database



4. Add database to the project

```
.env hr-management-system\.env
17 LOG_CHANNEL=stack
18 LOG_STACK=single
19 LOG_DEPRECATIONS_CHANNEL=null
20 LOG_LEVEL=debug
21
22 DB_CONNECTION=mysql
23 DB_HOST=127.0.0.1
24 DB_PORT=3306
25 DB_DATABASE=hrsys
26 DB_USERNAME=root
27 DB_PASSWORD=
28
29 SESSION_DRIVER=database
30 SESSION_LIFETIME=120
31 SESSION_ENCRYPT=false
32 SESSION_PATH=/
33 SESSION_DOMAIN=null
34
```

5. Run migrations

php artisan migrate

6. Create a model

php artisan make:model Employee-m

7. Add variables to the Model

```
use Illuminate\Database\Eloquent\Model;

class Employee extends Model
{
    use HasFactory;

    protected $fillable = [
        'name',
        'role',
        'salary',
        'age'
    ];
}
```

8. Update migrations

```
public function up(): void
{
    Schema::create('employees', function (Blueprint $table) {
        $table->id();
    });
}
```

```
        $table->string('name')->nullable();
        $table->string('role')->nullable();
        $table->decimal('salary',10,2)->nullable();
        $table->integer('age')->nullable();
        $table->timestamps();
    });
}
```

php artisan migrate

9. Create a controller

php artisan make:controller EmployeeController

10. Implement CRUD operation

```
public function store(StoreEmployeeRequest $request)
{
    Employee::create($request->validated());
}

public function all()
{
    return Employee::all();
}

public function get($id)
{
    return Employee::findOrFail($id);
}

public function edit($id)
{
    return Employee::findOrFail($id);
}

public function update(Request $request, $id)
{
    $employee = Employee::findOrFail($id);
    return $employee->update($request->all());
}

public function delete($id)
{
    $employee = Employee::findOrFail($id);
    return $employee->delete();
}
```

11. Implement routes

```
Route::get('/token', function () {  
    return csrf_token();  
});  
  
Route::post('employee', [EmployeeController::class, 'store']);  
Route::get('employee/all', [EmployeeController::class, 'all']);  
Route::get('employee/{id}/edit', [EmployeeController::class,  
    'edit']);  
Route::post('employee/{id}', [EmployeeController::class, 'update']);  
Route::delete('employee/{id}', [EmployeeController::class,  
    'delete']);
```

12. Test APIs

Discussion:

- Why do we use Postman?
- Why do we use Request Validation Class?